SAPIENZA
UNIVERSITÀ DI ROMA

Practical Network Defense a.y. 23-24

Assignment 3 Report - ACME 27 Group

Mija Simone Nicolae, 1895592 - mija.1895592@studenti.uniroma1.it
Palozzi Massimo, 2132174 - palozzi.2132174@studenti.uniroma1.it
Scarselli Ilaria, 1918975 - scarselli.1918975@studenti.uniroma1.it
Uffreduzzi Giorgia, 1668798 - uffreduzzi.1668798@studenti.uniroma1.it

October 31, 2024

SAPIENZA
UNIVERSITÀ DI ROMA

Contents

1 Introduction

The objective of this assignment was to enhance the security of the ACME network through a comprehensive hardening process. To achieve this, we structured our work around specific policies governing the control plane, data plane, and management plane. Key implementations included: a transparent proxy and reverse proxy to enforce web access control through policy-based routing; Graylog configured as a centralized SIEM solution for real-time monitoring and alerting, ensuring prompt detection of potential security incidents; Suricata IPS integrated into OPNsense on the main firewall to block malicious traffic and protect the network perimeter. We then further defined other specific policies across these planes, distributing the tasks among team members.

2 Control Plane

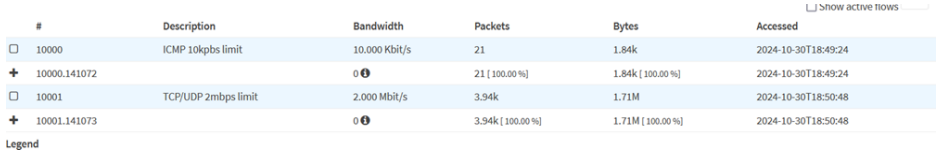
2.1 DDoS Mitigation with Traffic Shaping in OPNsense

To protect the ACME network control plane from potential DDoS attacks, we configured traffic-shaping rules on the Main Firewall WAN interface in OPNsense. These rules limit the rate of specific traffic types, reducing the risk that malicious, high-volume traffic could overwhelm the control functions of the network.
Our configuration includes the following limits:

- **ICMP (IPv4 and IPv6)** traffic is restricted to 10 kbps to mitigate ICMP flood attacks, which are often used to overload network resources.
- **TCP and UDP** traffic is capped at 2 Mbps each to prevent excessive connections and data packets from saturating the firewall processing capacity.

By setting these bandwidth limits, we effectively throttle incoming traffic that could otherwise consume control plane resources, ensuring that essential network functions remain responsive even under heavy load.

Testing To verify the effectiveness of this configuration, we monitored the traffic shaping status provided by OPNsense:

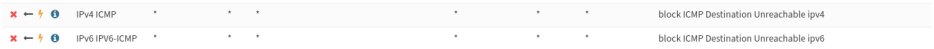

Show active flows

	#	Description	Bandwidth	Packets	Bytes	Accessed
☐	10000	ICMP 10kpbs limit	10.000 Kbit/s	21	1.84k	2024-10-30T18:49:24
+	10000.141072		0	21 [100.00 %]	1.84k [100.00 %]	2024-10-30T18:49:24
☐	10001	TCP/UDP 2mbps limit	2.000 Mbit/s	3.94k	1.71M	2024-10-30T18:50:48
+	10001.141073		0	3.94k [100.00 %]	1.71M [100.00 %]	2024-10-30T18:50:48

Legend

2.2 Filter Out ICMP Destination Unreachable Packets

Since some obsolete or potentially dangerous ICMP packets are already managed by the IPS, and ICMP Redirects were addressed in the first assignment, we decided to explicitly block all outgoing ICMP Destination Unreachable packets through a rule on the WAN interface. This action not only helps reduce CPU overload but also prevents revealing information about the internal network topology.



3 Data Plane

3.1 Squid Proxy

In the DMZ, a Squid-based proxy service has been implemented to efficiently manage network traffic. This open-source proxy server operates in two distinct modes: a forward proxy that listens on port 3127 for HTTP connections and on port 3126 for HTTPS connections. This setup enables secure access for external hosts to internal services. Additionally, a transparent proxy is configured on ports 3128 and 3129, which automatically intercepts HTTP and HTTPS requests from internal hosts to go to external services (EXTERNAL network and Internet). Squid also incorporate SSL Bump to handle HTTPS traffic and establish authentication rules to control user access.

3.1.1 Configuring the Squid Proxy

The first step in our configuration involved modifying the main Squid configuration file, located at `/etc/squid/squid.conf`.

Forwarding Mode

We configured the proxy for forwarding mode by specifying the ports on which it would listen for incoming traffic. For HTTP connections, we set up port 3127, while HTTPS connections were assigned to port 3126, complete with the necessary SSL certificate and key. Additionally, we ensured that the proxy was accessible via IPv6 by adding entries for port 3137 and 3136.

```
1    # Forwarding Mode
2    http_port 3127
3    http_port [::]:3137
4    https_port 3126 cert=/etc/squid/ssl_cert/squid.crt key=/etc/squid/ssl_cert/squid.
     key
5    https_port [::]:3136 cert=/etc/squid/ssl_cert/squid.crt key=/etc/squid/ssl_cert/
     squid.key
```

Transparent Mode

Next, we set up transparent mode. In this mode, Squid intercepts traffic from clients without requiring any explicit configuration on their part. We designated ports 3128 for HTTP and 3129 for HTTPS, also including SSL Bump functionality to manage encrypted connections.

```
# Transparent Mode
http_port 3128 intercept
http_port [::]:3138 intercept
https_port 3129 intercept ssl-bump cert=/etc/squid/ssl_cert/squid.crt key=/etc/squid/ssl_cert/squid.key generate-host-certificates=on
https_port [::]:3139 intercept ssl-bump cert=/etc/squid/ssl_cert/squid.crt key=/etc/squid/ssl_cert/squid.key generate-host-certificates=on
```

3.1.2 Managing HTTPS Traffic with SSL Bump

To effectively manage HTTPS traffic, we implemented SSL Bump. This feature allows Squid to decrypt, inspect, and re-encrypt secure traffic. We configured the necessary parameters, including the SSL certificate generation program and the rules for handling SSL connections.

```
# SSL Bump
sslcrtd_program /usr/lib/squid/security_file_certgen -s /var/lib/ssl_db -M 4MB

acl step1 at_step SslBump1
ssl_bump peek step1
ssl_bump bump all

sslproxy_cert_error allow BrokenButTrustedServers
sslproxy_cert_error deny all

tls_outgoing_options options=NO_SSLv3
```

3.1.3 Access Control Lists (ACLs) and Authentication

A critical aspect of our configuration involved setting up Access Control Lists (ACLs) and authentication rules to ensure that only authorized users could access the proxy.

Defining ACLs

We began by defining the local networks that would be permitted to use the proxy. This included various ranges, such as private networks and specific internal subnets. We also defined ACLs for users who could access the proxy in both forwarding and transparent modes.

```
acl localnet src fc00::/7                 # RFC 4193 local private network range
acl localnet src fe80::/10                # RFC 4291 link-local (directly plugged) machines
acl localnet src 2001:470:b5b8:1b04::/64  # DMZ
acl localnet src 127.0.0.0/8              # Loopback address
acl localnet src 100.100.6.0/24           # Specific internal subnet
```

```
6
7     acl forward_users src 100.100.4.0/24
8     acl forward_users src 100.101.0.0/24
9     acl forward_users src 2001:470:b5b8:1b04::/64
10
11    acl transparent_users src 100.100.2.0/24
12    acl transparent_users src 100.100.1.0/24
13    acl transparent_users src 2001:470:b5b8:1b82::/64
14    acl transparent_users src 2001:470:b5b8:1b81::/64
15
16    acl BrokenButTrustedServers dst 100.101.0.4
```

Implementing Authentication

To further secure our proxy, we implemented basic authentication. This required users to authenticate before accessing the proxy. We set up an external authentication program, using a password file named `squid_passwd`, which contained only one entry : `acme:k8NwA7fP3xZq` In our configuration, we established an ACL for authenticated users and defined the authentication program as follows:

```
1     acl authenticated proxy_auth REQUIRED
2     auth_param basic program /usr/lib/squid/basic_ncsa_auth /etc/squid/squid_passwd
3     auth_param basic realm Proxy Squid
```

Finally, we set access rules to manage who could connect to the proxy:

```
1     http_access allow localnet
2     http_access allow transparent_users
3     http_access allow forward_users authenticated
4     http_access deny all
```

These rules ensured that only users from defined networks and authenticated users could access the proxy, providing a robust security layer.

3.1.4 Restarting Squid

After completing our configuration, we needed to restart the Squid service to apply the changes. This was done with the command:

```
1     sudo systemctl restart squid
```

3.1.5 Verifying the Configuration

To ensure everything was functioning correctly, we checked the status of the Squid service:

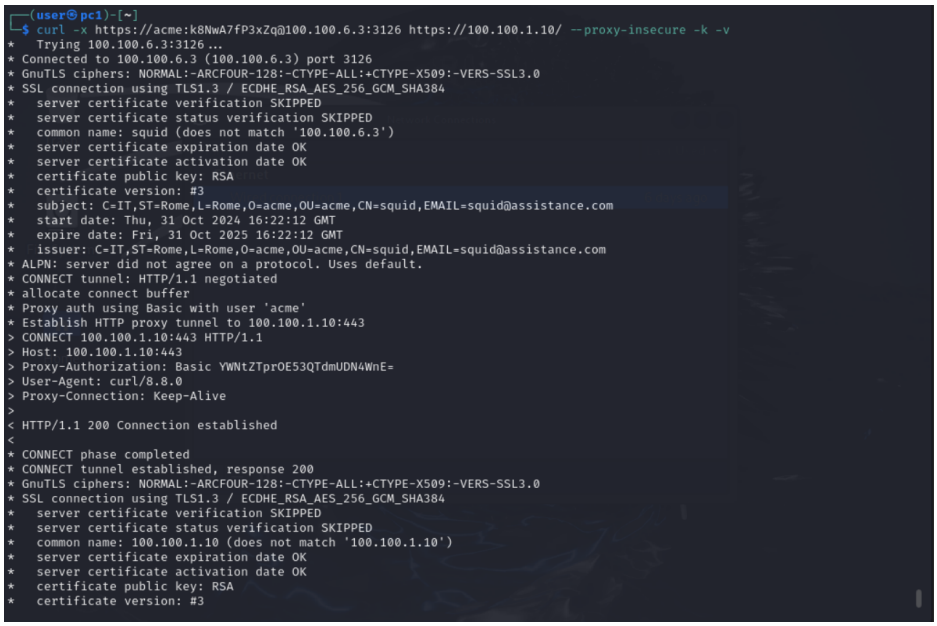
```
1     sudo systemctl status squid
```

We also monitored the logs to look for any errors or issues:

```
1     sudo tail -f /var/log/squid/access.log
```

3.1.6 Client Configuration

For clients utilizing the forwarding mode (i.e. clientext1 machine), we instructed them to connect to the Squid server using its IP address and port 3127 (or 3126 for HTTPS connection). In the case of transparent mode, internal hosts (hosts in CLIENTS and SERVERS networks) were able to access the proxy seamlessly without needing to configure any specific settings or authentication, as they are considered "authenticated" by the server. This simplifies access for internal users while maintaining security for external connections.



3.2 Reverse Proxy

It was necessary to implement a reverse proxy on the Apache web server in the DMZ with the function of SSL endpoint for the "Fantastic Coffee Machine" server, located in the EXTERNAL network, which does not natively support HTTPS. The main goal of this configuration was to ensure that communications between users and Fantastic Coffee server are secure. This solution not only protects information during transmission but also improves server accessibility through a centralized interface managed in the web server.

3.2.1 Enabling Required Modules

Initially, we enabled the required modules on Apache by executing the following commands:

```
1    sudo a2enmod proxy
2    sudo a2enmod proxy_http
3    sudo a2enmod ssl
```

These modules are essential for managing HTTPS traffic and ensuring that requests are correctly forwarded to the backend server.

3.2.2 Creating the Virtual Host

A configuration was added to the file for the Virtual Host "webserver.acme-27.test.conf" in the /etc/apache2/sites-available/ directory:

```
<VirtualHost *:443>
    ServerName reverse.proxy

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/fantasticcoffee.crt
    SSLCertificateKeyFile /etc/ssl/certs/fantasticcoffee.key

    ProxyPreserveHost On
    ProxyPass / http://100.100.4.10/
    ProxyPassReverse / http://100.100.4.10/

    ErrorLog /var/log/apache2/reverse-proxy-error.log
    CustomLog /var/log/apache2/reverse-proxy-access.log combined
</VirtualHost>
```

3.2.3 Generating Certificates

1. **Generate a Certificate from the Certificate Authority (CA):** First, a CA certificate was created to sign the server certificates.

```
# Generate the private key for the CA
openssl genrsa -out ca.key 2048

# Generate the CA certificate
openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -out ca.crt -subj
"/CN=My Self-Signed CA"
```

2. **Generate a Certificate for the Server** After creating the CA, a server certificate was generated and signed with the CA. Here are the steps:

```
# Generate the private key for the server
openssl genrsa -out fantasticcoffee.key 2048

# Create a certificate signing request
openssl req -new -key fantasticcoffee.key -out fantasticcoffee.csr -subj "/CN=
reverse.proxy"

# Sign the server certificate with the CA
openssl x509 -req -in fantasticcoffee.csr -CA ca.crt -CAkey ca.key -
CAcreateserial -out fantasticcoffee.crt -days 365
```

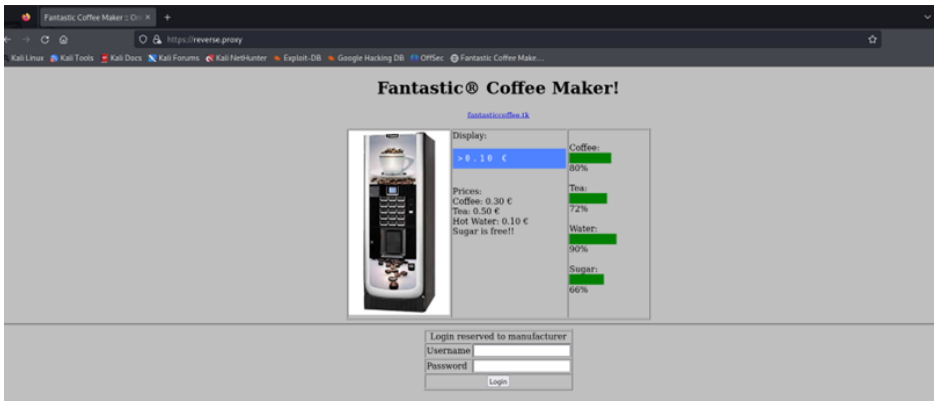
3.2.4 Enabling the Site and Restarting Apache

Once the Virtual Host was configured, we enabled the site and restarted the Apache server to apply the changes.

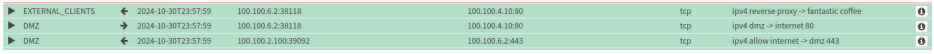
```
sudo a2ensite webserver.acme-27.test.conf
sudo systemctl restart apache2
```

3.2.5 Testing the Configuration

To test the configuration, access the domain reverse.proxy via HTTPS in a browser (i.e. from Kali machine), and verify that the content is loaded correctly from the Fantastic Coffee server.



In the Log View section in the opnsense Main Firewall:



3.3 LogServer and Graylog: from UDP to TCP

To enhance reliability, LogServer and Graylog will collect logs using TCP instead UDP. To do so, we once again changed the rsyslog configuration and the input configuration for graylog, enabling the TCP protocol. After this procedure, all hosts which send logs to graylog and logserver had to be reconfigurated to send TCP packets and not UDP packets.

4 Management Plane

4.1 Fail2Ban

We installed Fail2Ban on all machines across the ACME network to enhance management plane protection, excluding the firewalls which already had a built-in SSH lockout feature. This setup aims to

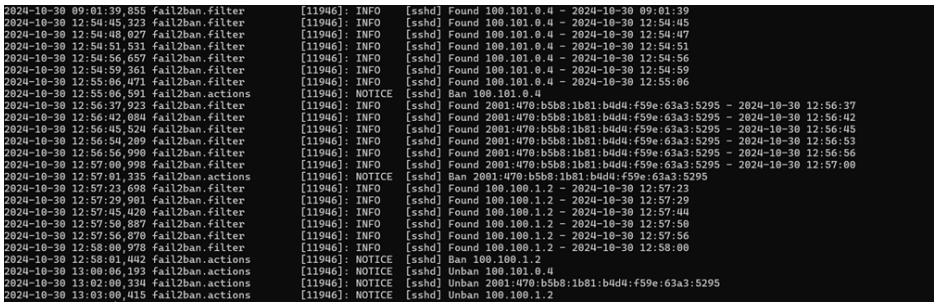
guard the management plane by automatically detecting and blocking SSH brute-force attacks, preventing unauthorized access to network administration systems.
To configure SSH brute-force protection, we launched the following command on each host:

```
echo '[sshd]\
enabled = true\
port = ssh\
filter = sshd\
backend = system\
maxretry = 6\
bantime = 300\
findtime = 300' > /etc/fail2ban/jail.local; systemctl enable fail2ban; systemctl
restart fail2ban
```

This configuration, written to /etc/fail2ban/jail.local, establishes the following parameters:

- **maxretry**: Limits the maximum number of failed attempts to 6 within the **findtime** window.
- **bantime**: Sets a temporary ban period of 300 seconds (5 minutes) for IP addresses exceeding the maximum retries.
- **findtime**: Defines a 5-minute window to count login attempts.

Testing To confirm the effectiveness of our Fail2Ban setup, we generated multiple failed login attempts to trigger a ban. As expected, Fail2Ban blocked further access attempts after reaching the configured threshold. The logs confirmed that the ban was enforced, and after the specified 5-minute period, the IP address was automatically unbanned.



4.2 Passwords

We strengthened management plane security by updating all passwords to complex, unique credentials, ensuring robust access control. This change, combined with Fail2Ban brute-force protection, effectively reduces the risk of unauthorized access and enhances overall security for ACME administrative interfaces.

Host	Credentials
webserver	root:k8NwA8hbWnaG54OO
proxyserver	root:KR5lomr17yP0DhvC
dnsserver	root:kd3byvWy0L9v8AhX
graylog	root:isH3YH1mgsHWt5iz
graylog-GUI	admin:T9kL3jX2pR8w
greenbone	root:2A7Fvz02NkVQLZhL
greenbone-GUI	admin:M5zQ1vN4yH7s
logserver	root:P8vfyRAp4ThW7KCt
kali	user:Mz30H9Nxkf2QWqle
arpwatch-clients	root:bYNu7hXkmj3Ez4lK
client-ext-1	user:pO1zhvSK8LbmG4eA
main-firewall	root:Ot8mz3XuwFY9r1B7
internal-firewall	root:N3quLc9YaEWiZ8F1

4.3 HTTPS to log in - Firewall

Through System ▷ Settings ▷ Administration, we configured HTTPS as the protocol for securely accessing the web GUI of both firewalls, instead of the previously used HTTP.

4.4 HTTPS to log in - Graylog

To allow HTTPS on Graylog, we firstly created a private key and a self signed certificate.

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/graylog/pkcs5-
    plain.pem -out /etc/ssl/graylog/cert.pem /subj "/CNN=100.100.1.10"
```

Since Graylog supports only PKCS8 keys, we had to convert the key generated (which is PKCS5) into PKCS8. We once again used openssl.

```
openssl pkcs8 -in /etc/ssl/graylog/pkcs5-plain.pem -topk8 -nocrypt -out pkcs8-plain.
    pem
```

After doing that, we configured Graylog to use only https, with the new created self signed certificate, by adding the following parameter in the Graylog server file (/etc/graylog/server/server.conf).

```
http_enable_tls = true
http_tls_cert_file = /etc/ssl/graylog/cert.pem
http_tls_key_file = /etc/ssl/graylog/pkcs8-plain.pem
```

The ACME Industry will change the self signed certificate with a Trusted one (no, they will forget about it and release it in production, creating another mess like i almost this a couple of days ago at work).

5 Network Time Protocol

To set up an NTP service, we first configured both firewalls. We set the Main Firewall to sync with external NTP servers, then act as the time server for all its directly connected subnets and the

Internal Firewall. The Internal Firewall, in turn, was configured as a client of the Main Firewall and a server for its directly connected networks. Under Services ▷ Network Time ▷ General, we enabled all interfaces for both devices and then on the internal firewall we specified only the IPv4 and IPv6 addresses of the Main Firewall as sources. Next, we enabled incoming traffic for both IPv4 and IPv6 directed to the firewalls themselves on port 123 for NTP on all interfaces except the WAN. This way, we prevent direct NTP requests from external sources, but we still allow responses to the Main Firewall outgoing NTP requests, as they are part of an established connection.
We then configured all other hosts to act as clients of their respective firewalls (so, hosts in the internal subnets will send requests to the Internal Firewall). We used **ntpd** for most hosts, except for Kali, Client-ext, and Greenbone, which use **ntpsec**. In any case, the configuration process is similar, as it requires adding the IPv4 and IPv6 addresses of the intended firewall into the configuration file, with a line similar to ***server 100.100.1.1 iburst***. Once the service is started, we can obtain the following result by using the command ***ntpq -p***:

```
root@dnsserver:/etc# ntpq -p
     remote           refid      st t when poll reach   delay   offset  jitter
==============================================================================
 0.debian.pool.n .POOL.          16 p    -   64    0    0.000   +0.000   0.000
 1.debian.pool.n .POOL.          16 p    -   64    0    0.000   +0.000   0.000
 2.debian.pool.n .POOL.          16 p    -   64    0    0.000   +0.000   0.000
 3.debian.pool.n .POOL.          16 p    -   64    0    0.000   +0.000   0.000
-ns2.fiberteleco 195.32.69.63     2 u   38   64  177   26.186   -5.113   0.172
-31.14.133.122 ( 13.161.101.224   2 u   32   64  177    5.924   -0.016 116.781
+93-44-243-48.ip 193.204.114.233  2 u   36   64  177    4.839   -0.079   3.602
+acacia.bilink.n 193.204.114.232  2 u   34   64  177   10.364   +0.178   0.167
*ntp2.inrim.it   .CTD.            1 u   31   64  177   11.795   -0.004   0.756
-188.213.165.209 193.204.114.232  2 u   32   64  177    5.776   -0.130 116.796
-95.110.254.234  13.161.101.224   2 u   33   64  177   11.487   -3.082   1.780
+ntp2.leontp.com .GPS.            1 u   31   64  177   31.869   -0.106   3.519
-ns1.fiberteleco 192.168.10.1     2 u   36   64  177   26.201   -5.264   0.201
```

After enabling and restarting the service:

```
root@arpwatch-clients:~# ntpq -p
     remote           refid      st t when poll reach   delay   offset  jitter
==============================================================================
*100.100.2.1     100.100.254.1    3 u   15   64   37    0.278   +0.616   0.052
+internal-firewa 100.100.254.1    3 u   17   64   37    0.816   +0.889   0.207
```

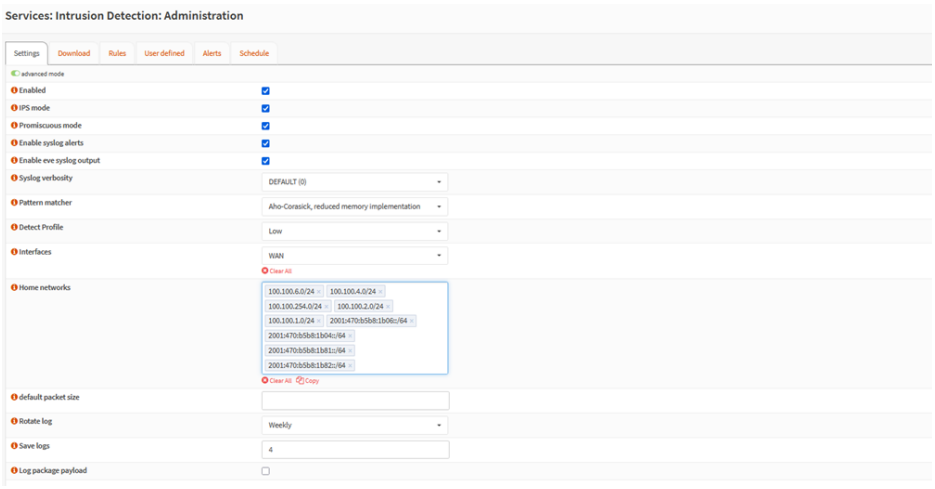
Finally, after a few polling cycles, the host will have stabilized and selected its preferred server (indicated by an asterisk). As shown here, the display is essentially the same for ntpsec as well, and here we can see that the Main Firewall is using an external server as source:

```
┌──(root㉿pc1)-[/home/user]
└─# ntpq -p
     remote                                refid      st t when poll reach   delay   offset  jitter
==============================================================================
+100.100.4.1                               85.199.214.99    2 u    -   64    1   0.3980   5.0569   0.0563
 2001:470:b5b8:1b04:d42d:f4ff:fe06:e2d9    85.199.214.99    2 u    9   64    1   0.4489   5.0445   0.0000
```

6 Cross-plane Aspects

6.1 Intrusion Prevention System

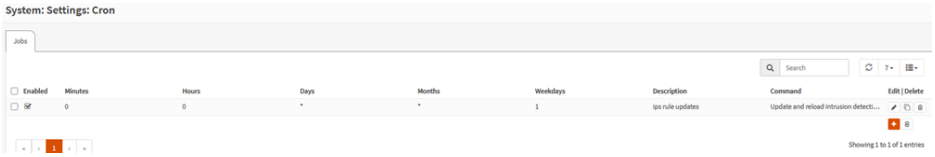
We integrated an Intrusion Prevention System (IPS) to strengthen the ACME network's security across all three planes: management, control, and data. We decided to configure the IPS only on the Main Firewall on the WAN interface: initially, we tried inspecting all the interfaces, but the overhead was to big, and the firewall couldn't keep up; therefore, we chose to capture traffic only on the WAN interface as it is the choke point of the ACME Network.
To further reduce overhead, we used "Aho-Corasick, reduced memory implementation" as a pattern matcher and a low detect profile.



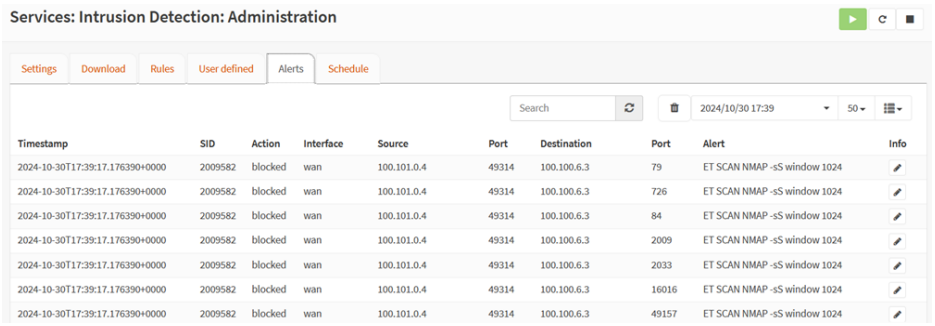
Next, we needed to enable specific rules, with the option to choose between manual and automatic configurations. We opted for automatic rules due to their ease of use, reliability, and the added benefit of being easily updatable. The few selected rules provide broad coverage for multiple threat types, reducing the risk of overload on the firewall. The rulesets we enabled are:

- **ET open/emerging-scan**: Protects against common scanning behaviors.
- **ET open/emerging-malware**: Targets known malware traffic signatures.
- **ET open/emerging-icmp**: Manages ICMP traffic to prevent misuse.
- **ET open/emerging-exploit**: Detects exploit attempts targeting vulnerabilities.
- **ET open/emerging-dos**: Helps mitigate Denial-of-Service attacks.

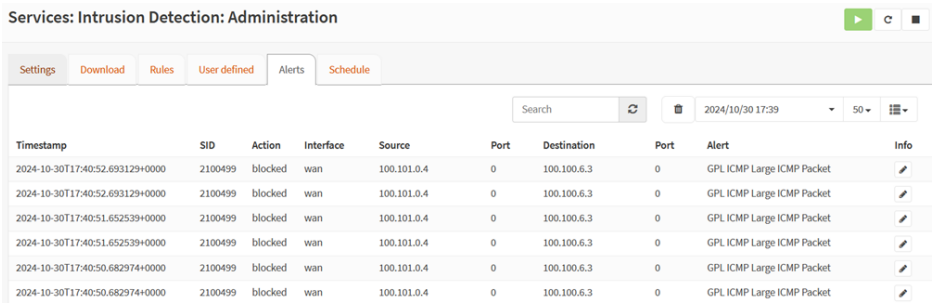
Then, we set up a cronjob to fetch and update all rules every Monday.



Finally, under **Service ▷ Intrusion Detection ▷ Policies**, we configured policies for the downloaded rules. We opted to block all traffic flagged as suspicious by the IPS, prioritizing security with an aggressive stance against potentially harmful connections. This decision, however, led to some challenges: for instance, our Main Firewall could no longer obtain an IPv6 address due to blocked traffic. To resolve this, we had to manually adjust certain rules to allow necessary IPv6 traffic while still maintaining a high level of security.
To test the IPS and validate its effectiveness, we used tools like Nmap to perform network scans and attempted to send large ICMP packet transmissions. These tests helped us assess the IPS ability to detect and block various types of suspicious activity.
Nmap:



Large ICMP:

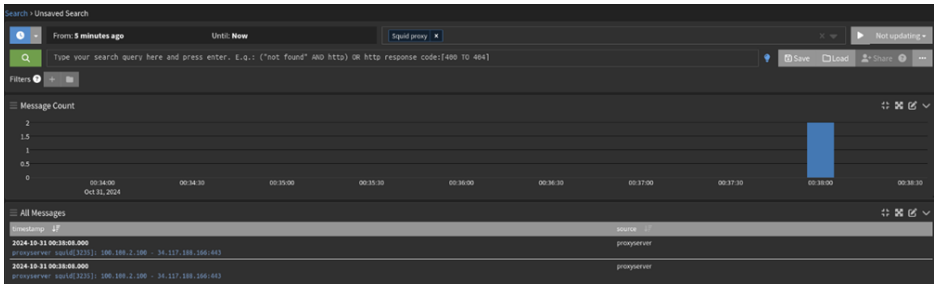


6.2 SIEM Reporting & Dynamic Fail2Ban

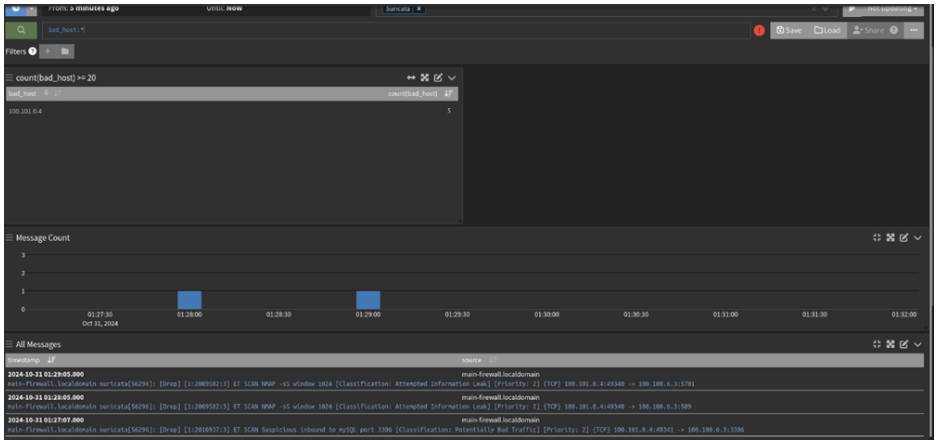
We implemented a SIEM solution using Graylog for real-time monitoring and integrated Fail2Ban for dynamic IP blocking, enhancing ACME network security.

6.2.1 Stream Configuration

1. **Transparent Proxy HTTP/HTTPS Stream**: Created a stream to monitor HTTP/HTTPS traffic through ACME transparent proxy, enabling visibility into web activity and the detection of unauthorized access attempts.

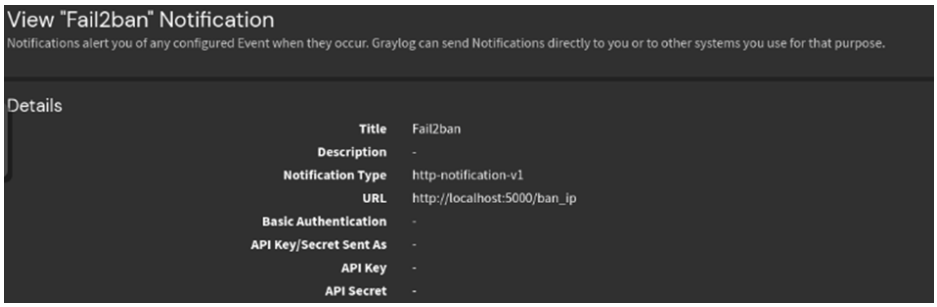


2. **Suricata Stream on Main Firewall**: Configured a stream for Suricata alerts on the Main Firewall. When an IP generates 20 or more alerts, Graylog triggers an automatic response that bans the IP across all ACME network components. Additionally, we also added an extractor that used a regex to extract *bad_host* to ban.



6.3 Automated Banning with Fail2Ban

To enable automated bans, we developed a Python script that runs on a server listening on localhost. When the 20-alert threshold is met, Graylog sends a request to this local server. The script then interacts with the OPNsense API to add the flagged IP to a ban list, effectively blocking it across the network.



Finally, we added an alias called "local_fail2ban" along with a rule on the WAN interface to block any traffic coming from those IPs.

Testing As before, conducting multiple Nmap scans will eventually trigger the 20-alert threshold, resulting in the IP being banned from accessing the ACME network. We ensured that banned IPs appeared showed up with "local_fail2ban" alias on the Main Firewall.

7 Additional Unimplemented Considerations

There are several other approaches we considered but ultimately discarded in favor of the ones just discussed, due to various reasons. We will provide a brief description below.

7.1 Multifactor authentication

Multifactor authentication (MFA) requires users to provide two or more verification factors to gain access to an account or system. This adds an extra layer of protection, reducing the risk of identity theft and data breaches, but with the cost of complicating the system.

7.2 Updater

Another step that can be implemented is the configuration in the SIEM of an updater that periodically keeps the various hosts connected to the network up to date.

7.3 Honeypots

Honeypots are designed to lure attackers by simulating vulnerable systems or files (as a monitored file that is not typically accessed by authorized users). They act as decoys to detect, analyze, and study malicious activity, providing insights into attack methods and enhancing overall security while helping to distract attackers from actual targets.

7.4 Layer 2 Protection and Spoofing Prevention

For example, disabling gratuitous ARP packets, enabling Dynamic ARP Inspection, enabling port security, enabling DHCP snooping. These are all measures applicable at Layer 2, which is minimally or not managed at all by OPNsense. For this reason, they should be addressed at the switch management level, where there are directly implementable options to handle them without adding too much complexity. While ARP tables can be viewed as an aid for IP source guard, IP spoofing has been addressed directly through a combination of other mechanisms described earlier.

7.5 Authentication Methods for Routing Protocols

They enhance the security of routing updates by verifying the identity of routers communicating within a network, ensuring that only authorized routers can participate in routing exchanges, thereby preventing unauthorized access, route manipulation, and ensuring the integrity of routing tables.

7.6 iptables on the Hosts

Implementing iptables on the various hosts provides an additional layer of security by allowing fine-grained control over incoming and outgoing traffic. In the end we preferred a more centralized approach.

7.7 Allowing only HTTPS traffic and blocking HTTP

While this can be potentially very beneficial for security, it could lead to significant overload or be overly restrictive.

8 Conclusions

We're really pleased with the final setup, which aligns well with both our current security goals and the policies from previous assignments. Testing each component was challenging, particularly with the IPS occasionally blocking our requests and a persistent DHCP-PD bug in OPNsense, but seeing everything function as intended made the effort worthwhile. We also considered adding a few extra measures, like MFA for sensitive management interfaces, Layer 2 protections (discarded since we couldn't configure switches), honeypots for early threat detection, and expanded SIEM reporting to increase network visibility. Ultimately, we prioritized the most critical security needs, leaving these extra enhancements as potential future improvements.